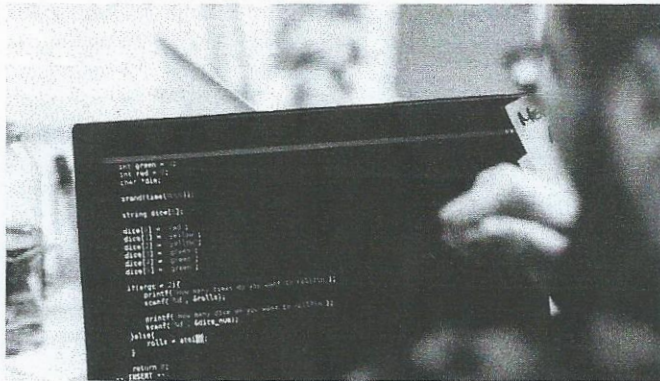


15 PROGRAMMING

Programming is the process of creating a set of instructions, or computer programs, using computer programming languages. A program is therefore a sequence of written instructions which tell a computer how to perform operations and specific tasks. We can program almost everything, e.g. invent a game, control a robot or solve a problem.



Programmer

Programming includes activities such as selecting a language, analysing, generating algorithms, writing source code (coding), compiling (converting into machine code), testing, debugging, developing and maintaining the program.

Most programming languages are based on the **concept of a statement** – an instruction expressing an action to be executed by the computer and containing expressions. The structure and composition of statements is called **syntax**. There is a variety of languages which use different syntax. The order in which statements are executed in a program is called **control flow**. Programming languages specify identifiers (entities), data types, subroutines (functions) and the like. An **identifier** is a name given to an entity; it can be declared or defined. A **declaration** specifies the name and data type of a variable or other entity. A **definition** also allocates memory for an identifier, so it must be defined only once but it can be declared many times. A **variable** is a storage area (placeholder) which holds data and is given a name (identifier). A **data type** determines kind of a value, range, operations held by a variable, etc. Basic

data types supported by most computer languages are numeric (integer, floating-point numbers), Boolean (logic), character, date, string, or array data type.

Programming paradigms

There are different styles of programming, called programming paradigms, meaning ways how programs are written. Programs can be written using a sequence of commands (control flow) – which is called *imperative programming*. A kind of imperative programming is *structured programming* where the control flow is defined by conditions, loops, subroutines, etc.

Object-oriented programming is based on objects.

Declarative programming, in contrast to imperative, focuses on result without using control flow or describing the steps. *Visual (or block-based) programming* uses a visual programming environment where a program is represented and manipulated as graphical objects and symbols (graphic blocks) without dealing with textual syntax.

In a **structured programming** programs are written in a high-level language based on a series of structures (instructions and procedures) containing logical steps. Control flow statements are divided into three kinds:

- 1) *order of sequences* (the ordered sequencing of commands)
- 2) *branching* (conditional statements depending on whether a condition evaluates to true or false): if...then, if...then...else, case and switch statements
- 3) *loops*: In a loop structure a process is repeated as long as certain conditions remain true. Looping statements are e.g. WHILE loop, DO WHILE loop, FOR loop, etc.

In an **object-oriented programming** (OOP) programs are made from combinations of predefined modules which are reusable, so they can be easily reused. Object is a module containing data and program instructions that has its own properties created by bundling data and program instructions together. Objects can be classes, subclasses or instances. The programmer gives each object (e.g. a piece of text, a graphic, a table) functions which can be altered without changing the entire program. A class

is a set (collection) of objects of the same kind; it holds variables – properties and methods. Classes are made from data and properties (attributes, class variables) and methods (meaning 'do something': functions and procedures). Access modifiers help to implement encapsulation, e.g. public, private and protected. OOP enables to handle multimedia applications, develop graphical user interfaces (user interface design), simulations and modelling, object-oriented databases, etc.

The key features and concepts of OOP are:

- **Data Abstraction or Data Hiding**
- **Encapsulation** – property that allows bundling data and program instructions into modules called objects, to create complex applications.
- **Inheritance** – means that specific objects of a class inherit all the properties of the class (properties in a class are inherited by their subclasses and instances).
- **Polymorphism** – property that enables different objects to receive the same instructions but deal with them in different ways.
- **Code reusability** – means that new programs can easily be copied and pasted together from old programs.

VOCABULARY

access modifier /'ækses ,mɒd.ɪ.fai.ə/	přístupový modifikátor
algorithm /'æl.gə.rɪ.ðəm/	algoritmus
branching /brɑːntʃɪŋ/	větvění
bug /bʌg/	chyba
bundle /'bʌn.dəl/	zabalit
character (data type) /'kær.ək.tər/	znak (znakový datový typ)
class /klaːs/	třída
command /kə'mɑːnd/	příkaz
condition /kən'dɪʃ.ən/	podmínka
conditional statement /'kən'dɪʃ.ənəl ,stet.mənt/	podmíněný příkaz
control flow /kən'trəʊl ,fləʊ/	řídící struktura
control flow statements /'stet.mənt/	řídící struktury pro zápis programu
debug /,diː'bʌg/	ladit (program), odstraňovat chyby
debugging /,diː'bʌgɪŋ/	odstranění chyb
data type /'deɪ.tə ,taɪp/	datový typ
encapsulation /ɪn,kæpsjuː'leɪʃn/	zapouzdření
flowchart, flow chart /'fləʊ.tʃɑːt/	vývojový diagram
identifier /aɪ'dentɪfaɪər/	identifikátor
inheritance /ɪn'her.ɪ.təns/	dědičnost
integer (data type) /'ɪn.tɪ.dʒər/	celé číslo (celočíslný datový typ)
loop /luːp/	smyčka
object /'ɒb.dʒɪkt/	objekt

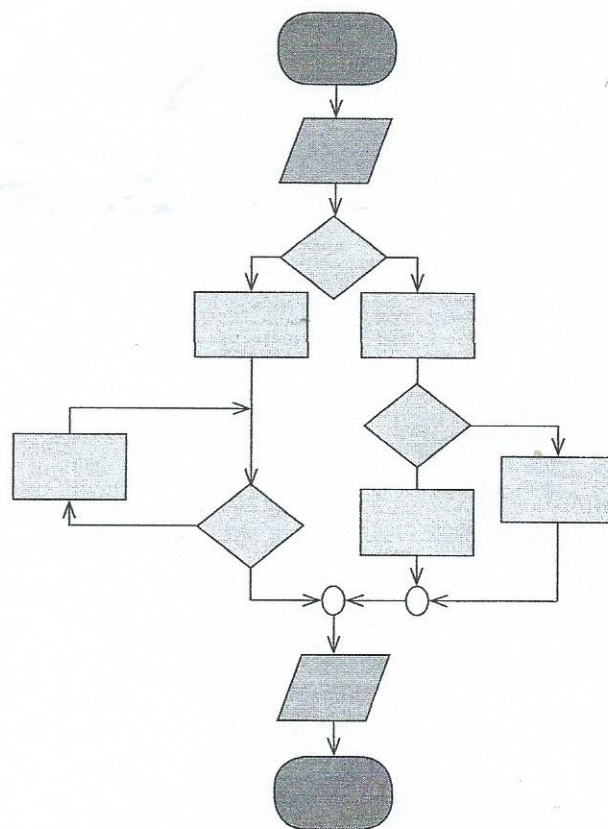
object-oriented programming /'ɒbdʒɪkt ɔːrɪəntɪd ,prəʊ.græm.ɪŋ/	objektově-orientované programování
order of sequences /'ɔːdə ɒv ,siːkwənsɪz/	posloupnost příkazů
paradigm /'pær.ə.daɪm/	paradigma
polymorphism /'pɒl.i'mɔːfɪ.zəm/	vícetvarost
property /'prop.ə.ti/	vlastnost
statement /'stet.mənt/	příkaz
string /strɪŋ/	řetězec
structured programming /'strʌk.tʃəd ,prəʊ.græm.ɪŋ/	strukturované programování
variable /'veə.rɪ.ə.bəl/	proměnná

► TASKS ◀

TASK 1

A basic flowchart

A flowchart is a type of diagram that shows the steps in a process. Match flowchart symbols (see picture Flowchart diagram) with their expressions and translate them:



The terminal or ellipse = the START/END symbol

The parallelogram = the INPUT/OUTPUT symbol or DATA

The diamond shape = the DECISION symbol

The rectangle = the OPERATION or PROCESS symbol

The circle = the CONNECTOR symbol

The arrow